

Fiori Elements 開發課程 6：擴充 SAP 標準 App (Adaptation Project)

環境：BTP ABAP Environment Trial (全新獨立專案 `fe06.travel.ext`，擴充標準 App `Demo App for Travel V2`；同一個 BTP 系統，不是另一套環境)

Lecture

這一課要解決的問題

fe01 ~ fe05 做的事情，本質上都是「自己開發一個全新的 App」——CDS View / BDEF / Service Binding 是我們自己建的 (rc05 / rc08)，前端專案也是我們自己 generate 出來的 (`fe01_connection_test`)。但對大多數 ABAPer 而言，真正常見的日常工作是另一件事：**SAP 已經交付了一個標準 Fiori App** (例如「**Manage Sales Orders**」「**Track Sales Orders**」)，客戶想加一個欄位、加一顆按鈕、掛一段自訂邏輯，但完全不能去動 SAP 自己維護的物件 (這正是 `CLAUDE.md` 第一條硬性規則：「不可修改 SAP 標準物件」)。這一課要學的就是 SAP 官方給這個情境的正式解法：**Adaptation Project**。

官方文件先查證：這是完全不同的一套機制，不是「fe01 精靈的變形」

查證 `SAP-docs` (`btp-fiori-tools / sap-help`) 確認，Adaptation Project 的正式定位是：在不修改原始 App 的前提下，建立一個「Application Variant」疊加在標準 App 上面——概念上跟 rap/CDS 課程學過的 Metadata Extension「疊加不修改原始物件」是同一種哲學，只是這次疊加的是前端層。

兩個查證出來的關鍵限制，做這一課之前要先知道：

1. 目標 App 必須被明確標記為「已釋出可供 Adaptation Project 擴充」——不是任何標準 App 都能拿來練習，SAP / 開發者要主動開放這個權限。
2. 部署 (Deploy) 這一步，目標套件如果是「ABAP for Cloud Development」(也就是我們一路在用的 BTP Trial 那種套件)，官方文件明講不支援部署——這代表這一課能做到「產生 Application Variant、Preview 看效果」，但「真的部署回系統讓其他使用者看到」這一步在我們的環境上很可能卡關，這一課沒有實測 Deploy (留給讀者，見「思考題」)。

這兩個風險都是先查文件才知道要注意，不是憑空猜的——這正是這門課一路強調的「查證優先」習慣。

🚩 第一個好消息：BTP Trial 上真的有 App 開放擴充，不用換系統

原本擔心 BTP Trial (開發者導向、精簡環境) 可能沒有任何 App 符合「已釋出擴充」的條件，需要換到 on-premise 系統才能練習。實測直接推翻了這個擔心：Command Palette 執行 `Fiori: Open Adaptation Project Generator`，System 選既有的 `TRL` 連線 (跟 fe01 建立的那筆一模一樣，不用重新輸入 URL / 登入)，Application 欄位點開後，清單裡列出了好幾個真實可擴充的標準 App：

```
Application Logs (nw.core.applogs, F1487, BC-SRV-APS-APL)
Demo App for Travel V2 (sap.demo.travel.v2)
Display Business Event Logs (s4.cfnd.displaybusinesseventlogs, F6021, CA-GTF-BEL)
Display Business Events by Objects (s4.cfnd.displaybusinessprocesslogs, F6035, CA-GTF-BEL)
Display Changes to Objects (nw.cfnd.displaychangeinformation, F8211, CA-GTF-BEL)
Maintain Job Users (nw.core.appjobs.users, F6156, BC-SRV-APS-APJ)
```

這份清單本身就是「查 App 有哪些」最直接的方法——每個項目後面括號裡的技術 ID / Fiori Catalog 代碼 (如 `F1487`) / Business Component (如 `BC-SRV-APS-APL`)，不用另外查 SAP Fiori Apps Reference Library 就能先掌握基本資訊。這一課選 `Demo App for Travel V2` (`sap.demo.travel.v2`)——這是 SAP 官方的旅遊示範 App，openSAP「Build Apps with RAP」課程常用的標準案例，資料結構 (Travel / Booking) 簡單好懂。

建立 Application Variant

Project Attributes 頁面填的欄位，命名規則明顯不是我們自訂的 `feNN_XXX` 慣例，而是 SAP 的「Application Variant」技術命名 (範例預設值是 `app.variant`)：

欄位	填的值
Project Name	fe06.travel.ext
Application Title	FE06 標準 App 擴充練習
Namespace	customer.fe06.travel (customer. 前綴是慣例，代表「非 SAP 自己的擴充」)
Project Folder Path	../src/ABAP_Training_Fiori_Elements (跟 fe01 ~ fe05 放同一層)
Enable TypeScript	No

⚠ 這個專案的資料夾名稱本身就是 **fe06.travel.ext** (點記法)，不是我們慣用的底線命名 **fe06_xxx**——這不是疏忽，是 Adaptation Project 工具的技術命名要求 (Application Variant ID 本身就非得用點記法)，這一課是這門課至今唯一一個資料夾名稱跟慣例不一致的例外。

按 Finish 產生的專案結構，跟 **fe01 ~ fe05** 完全不同：

```
fe06.travel.ext/
├─ package.json
├─ ui5.yaml
└─ webapp/
   └─ manifest.appdescr_variant ← 不是 manifest.json!
      └─ i18n/i18n.properties
```

manifest.appdescr_variant：一份「差異清單」，不是完整描述檔

```
{
  "fileName": "manifest",
  "layer": "CUSTOMER_BASE",
  "fileType": "appdescr_variant",
  "reference": "sap.demo.travel.v2",
  "id": "customer.fe06.travel",
  "namespace": "apps/sap.demo.travel.v2/appVariants/customer.fe06.travel/",
  "version": "0.1.0",
  "content": [
    { "changeType": "appdescr_ui5_addNewModelEnhanceWith", "content": { "modelId": "i18n" }, "texts": {
      "i18n": "i18n/i18n.properties" } },
    { "changeType": "appdescr_ui5_addNewModelEnhanceWith", "content": { "modelId": "@i18n" }, "texts": {
      "i18n": "i18n/i18n.properties" } },
    { "changeType": "appdescr_ui5_setMinUI5Version", "content": { "minUI5Version": "1.148.7" } },
    { "changeType": "appdescr_app_setTitle", "content": {}, "texts": { "i18n": "i18n/i18n.properties" } }
  ]
}
```

reference: "sap.demo.travel.v2" 就是查 **OData Service / App** 出處最直接的欄位——這個 App Variant 是疊加在哪個標準 App 上面，一眼可見。**content** 陣列裡每一筆都是一個 **changeType**，只記錄「跟原始 App 比起來多了什麼」（這裡是加了 i18n Model、設定最低 UI5 版本、改標題），完全不像 fe01 的 **manifest.json** 那樣是一份完整描述。

package.json 也有兩個這門課沒見過的指令：

```
"scripts": {
  "start": "fiori run --open /test/flp.html#app-preview",
  "start-editor": "fiori run --open /test/adaptation-editor.html"
}
```

start-editor 開的是這一課的主角——**Adaptation Editor**，視覺化編輯標準 App 的工具。

Adaptation Editor：畫面結構與一個容易誤觸的陷阱

`npm run start-editor` 之後（這次自動選到 8081，因為 fe01 專案的 8080 還占用著），畫面三個區塊：

- 左側 **Outline**：完整元件樹（`Component > NavContainer > XMLView > ... > SmartTable > Table`），可以用 **Filter Outline** 搜尋框快速篩選節點；下方還有「**Filter Changes**」/ **變更歷史清單**（一開始是「No historic changes — This application was not modified yet」）。
- 中間：Demo Travel App 的真實畫面（不是 mock 資料，`Travels (4,136)` 是這個共用 Trial 系統上真實的資料筆數）。
- 右側：選了節點後顯示 **Quick Actions**（一份針對目前頁面類型整理好的建議操作清單）+ **Properties**（Control ID / Control Type）。

⚠️ 頂端有兩個模式切換：**UI Adaptation** 與 **Navigation**，容易不小心誤觸——**UI Adaptation** 模式下點畫面才是「選取控制項來編輯」；如果切到（或不小心點到）**Navigation** 模式，點畫面會變成「像一般使用者一樣操作 App」，直接導覽進到別的頁面（例如點一筆 Travel 資料就會鑽進它的 Object Page）。這一課實測就誤觸過一次——點擊沒有反應時，先檢查是不是模式切換錯了，不要懷疑選取功能本身壞掉。

⚠️ 實測推翻的假設：**Quick Actions** 清單是「依頁面類型」不是「依選取的控制項」

一開始以為選取不同的控制項（例如整個表格 vs 表格裡的工具列）會讓右側 Quick Actions 面板切換成對應的「表格專屬選單」——實測發現不是這樣：不管在 Outline 裡選 `TableToolB...`、`OverflowToolbar`，還是選到正確的 `Table` 節點本身（Properties 面板的 Control Type 正確顯示 `sap.m.Table`），右側面板標題始終是同一份「**LIST REPORT QUICK ACTIONS**」，沒有變成獨立的「**TABLE QUICK ACTIONS**」。

完整清單（往下捲動看到全部）：

```
Show Page Controller          ( 已經有 Controller 之後，"Add Controller to Page" 會變成這個 )
Add Custom Page Action
Enable "Clear" Button in Filter Bar
Disable Semantic Date Range in Filter Bar
Enable Variant Management in Tables and Charts
Change Table Actions
Change Table Columns
Add Custom Table Action
Add Custom Table Column      ( 灰階，見下方說明 )
Enable Table Filtering for Page Variants
```

結論：這份清單是依「目前在哪個頁面類型」（**List Report vs Object Page**）決定的固定清單，不是依照選了哪個控制項動態變化——選取控制項只影響右側 Properties 面板顯示的 Control ID / Control Type，不影響 Quick Actions 清單本身。Object Page 有自己另一套清單（`Add Controller to Page` / `Add Custom Page Action` / `Add Header Field` / `Add Custom Section` / `Enable Variant Management in Tables`.....），這一課只實作了 List Report 的部分，Object Page 的部分留給讀者練習（見「動手練習」）。

實作一：**Add Controller to Page**——跟 fe05 完全同一套機制，套用在標準 App 上

點 `Add Controller to Page`，跳出一個極簡對話框：只要填 **Controller Name**（自動加 `.js` 副檔名），按 **Create**。

⚠️ 按 **Create** 之後，檔案還沒真的產生——左下角變成「UNSAVED CHANGES」，右側 INFO CENTER 明講：「Note: The `TravellistExt` controller extension will be created once you save the change.」，要按上方工具列的磁片圖示（**Save**）才會真的寫入硬碟。存檔後，硬碟上多了兩個檔案：

```
webapp/changes/coding/TravellistExt.js
webapp/changes/id_1787107779647_326_codeExt.change
```

`TravellistExt.js` 的骨架，跟 fe05 學過的 `ControllerExtension.extend(...)` 是同一套語法，工具還自動幫忙寫好完整的中文（其實是英文）註解說明 `override` 底下可以放哪些生命週期 Hook：

```

sap.ui.define(
  ['sap/ui/core/mvc/ControllerExtension' /*, 'sap/ui/core/mvc/OverrideExecution'*/],
  function (ControllerExtension /*, OverrideExecution*/) {
    'use strict';
    return ControllerExtension.extend("customer.fe06.travel.TravelListExt", {
      // override: { onInit: function() {}}, onBeforeRendering: ..., onAfterRendering: ..., onExit:
    ... }
  });
});

```

[id_1787107779647_326_codeExt.change](#) 這份「變更描述檔」是這一課最關鍵的查證發現：

```

{
  "changeType": "codeExt",
  "reference": "customer.fe06.travel",
  "content": { "codeRef": "coding/TravelListExt.js" },
  "selector": {
    "controllerName": "sap.suite.ui.generic.template.ListReport.view.ListReport"
  }
}

```

⚠️⚠️ 重大發現：這個標準 App 掛的是 **sap.suite.ui.generic.template.ListReport.view.ListReport**，不是我們 fe05 用的 **sap.fe.templates.ListReport.ListReportController**——代表 Demo App for Travel V2 底層其實是舊式的 OData V2 **Smart Template Generic App**，不是新式 **sap.fe.templates**（OData V4 Fiori Elements）！這是一個查文件查不到、只有實測才會發現的真相：很多 SAP 標準交付的 App（尤其是歷史比較久的）仍然是 **Smart Template** 世代，同樣是「Controller Extension」這個概念，掛的目標 Controller 類別名稱因為底層框架世代不同而完全不同——手動改 **manifest.appdescr_variant** 要自己猜這個名稱很容易猜錯，用 **Adaptation Editor** 精靈操作，工具會自動偵測並填對，這正是為什麼「用官方精靈」比「憑印象手動改設定檔」可靠的原因。

驗證邏輯真的有掛上：直接編輯本機的 **TravelListExt.js**（一旦透過精靈產生，之後就是一個普通的 .js 檔案，可以像 fe05 一樣直接編輯，不需要每次都回精靈操作）：

```

override: {
  onInit: function () {
    MessageToast.show("customer.fe06.travel.TravelListExt 已掛上標準 Demo Travel App");
  }
}

```

存檔後重新整理 Adaptation Editor 的預覽畫面，**MessageToast** 真的跳出來了——證實這個 Controller Extension 已經真實掛在標準 App 上，跟 fe05 我們自己 App 的機制完全同一套底層邏輯，只是這次目標是別人維護的標準物件。

實作二：**Add Custom Table Column**——Fragment-based 的 View Extension

第一次點 **Add Custom Table Column**，發現它是灰階、點不動的。滑鼠移上去停留，tooltip 說明原因：

This action has been disabled because the table rows are not available. Please load the table data and try again.

——先按過篩選列的 **Go**，讓表格真的載入資料，這個動作才會被啟用。這是一個合理的前置條件（不是 bug）：新增欄位這個動作，工具需要先看到真實的資料列才能提供合理的預設值。

按 **Go** 之後再點一次，這次跳出對話框，要求輸入 **Column Fragment Name**（欄位標題的 Fragment）跟 **Cell Fragment Name**（每一列儲存格內容的 Fragment），各自自動加 **.fragment.xml** 副檔名。填了 **CustomColumnHeader / CustomColumnCell**，Create → Save，硬碟上產生四個檔案：

```
webapp/changes/fragments/CustomColumnHeader.fragment.xml
webapp/changes/fragments/CustomColumnCell.fragment.xml
webapp/changes/id_1787108683017_237_addXML.change    ( Column · targetAggregation: "columns" )
webapp/changes/id_1787108683018_239_addXML.change    ( Cell · targetAggregation: "cells" )
```

`CustomColumnHeader.fragment.xml` (工具自動產生的骨架，留了清楚的說明註解)：

```
<core:FragmentDefinition xmlns:core='sap.ui.core' xmlns='sap.m'>
  <!-- viewName: sap.suite.ui.generic.template.ListReport.view.ListReport -->
  <!-- controlType: sap.m.Table -->
  <!-- targetAggregation: columns -->
  <Column id="column-0e2d78e7" width="12em" hAlign="Left" vAlign="Middle">
    <Text id="column-title-0e2d78e7" text="New column" />
    <customData>
      <core:CustomData key="p13nData" id="custom-data-0e2d78e7"
        value='{ "columnKey": "column-0e2d78e7", "columnIndex": "13" }' />
    </customData>
  </Column>
</core:FragmentDefinition>
```

`CustomColumnCell.fragment.xml`：

```
<core:FragmentDefinition xmlns:core='sap.ui.core' xmlns='sap.m'>
  <!-- targetAggregation: cells -->
  <Text id="cell-text-b4ca15a5" text="Sample data" />
</core:FragmentDefinition>
```

對應的 `.change` 檔案 (Column 那筆)：

```
{
  "changeType": "addXML",
  "content": {
    "targetAggregation": "columns",
    "index": 13,
    "fragmentPath": "fragments/CustomColumnHeader.fragment.xml"
  },
  "selector": {
    "id": "sap.demo.travel.v2::sap.suite.ui.generic.template.ListReport.view.ListReport::Travel--responsiveTable",
    "idIsLocal": false
  }
}
```

這就是「加 **Extension View**」最直接的展現：Fragment 本身是完全標準的 `sap.m` XML (跟一般 UI5 開發沒有任何特殊語法)，`.change` 檔案的 `selector.id` 精準指到標準 App 裡那個表格控制項的穩定 ID (`...::Travel--responsiveTable`)，`content.targetAggregation` / `content.index` 決定要插入到哪個聚合、第幾個位置——這一整套「不改原始碼、用穩定 ID + Fragment 疊加」的精神，跟這門課從 fe01 就開始強調的「疊加不修改」哲學是一致的，只是這次疊加對象換成了別人的標準 App。

⚠ 重要澄清：「Enhance UI Annotation」在這裡不是 fe02/fe03 那種做法

fe02 / fe03 教過的「本機 `annotation.xml` 疊加」，前提是我們自己擁有這個 **CDS View 的 Metadata Extension** 權限 (RAP Cloud 課程建的 `ZI_RC05_NOTE`)。標準 App 的 CDS View 是 SAP 自己維護的物件，我們沒有、也不該去疊加它的 Annotation——這正是 `CLAUDE.md` 那條硬性規則的體現。

那「Enhance UI Annotation」這個需求在 Adaptation Project 底下該怎麼做？實測發現有兩層，容易混淆：

1. **Change Table Columns**：點下去彈出的是一般使用者也看得到的「**View Settings**」個人化設定畫面（Columns / Sort / Filter / Group 分頁），列出 **Travel ID / Agency ID / Customer ID / Starting Date / End Date / Booking Fee / Total Price**.....這些標準 App 的 **CDS View** 本來就已經定義好、只是預設沒顯示的既有欄位，勾選 / 取消勾選只是調整「這個 Application Variant 預設顯示哪些既有欄位」。這不是新增全新的 **Annotation**，是調整既有 **Annotation** 已經曝露的欄位可見度。
2. **Add Custom Table Column**（上面實作二）：這才是真正新增一個標準 App **CDS View** 裡原本不存在的全新欄位，做法不是寫 `@UI.lineItem` annotation，而是用 Fragment 塞一個全新的 UI 元素進去（畫面上看起來像多了一欄，但底層資料要嘛是寫死的、要嘛要在 Fragment 或 Controller Extension 裡自己用 Binding / 自訂邏輯算出來，不是官方 Annotation 機制原生支援的欄位）。

結論：對「別人維護的標準 App」而言，沒有一個機制讓你「像 fe02/fe03 一樣直接疊加一段全新 `@UI.*` annotation 定義」——你只能在既有欄位範圍內調整顯示（**Change Table Columns**），或用 **UI5** 原生的 **Fragment/Extension** 機制自己刻一個看起來像欄位的東西（**Add Custom Table Column**）。這是這一課最重要的概念釐清，直接呼應這門課從一開始就強調的「不可修改 SAP 標準物件」規則——UI 層的擴充工具設計，本質上就是繞著這條規則打造的。

學習目標

- 知道 Adaptation Project 是什麼、跟 fe01 ~ fe05「自建 App」的本質差異：不修改原始 App，疊加一個 Application Variant
- 能講出 Adaptation Project 的兩個關鍵前提限制：目標 App 要「已釋出擴充」、部署到 ABAP for Cloud Development 套件可能不支援
- 知道怎麼用 **Fiori: Open Adaptation Project Generator** 精靈連線、選擇目標 App、建立專案，理解 `manifest.appdescr_variant`（差異清單）跟 `manifest.json`（完整描述）的本質不同
- 認識 Adaptation Editor 的三大區塊（Outline / 畫面 / Quick Actions + Properties），知道 **UI Adaptation** 與 **Navigation** 兩種模式的差異
- 知道 Quick Actions 清單是依「頁面類型」固定，不是依「選取的控制項」動態變化
- 能操作 **Add Controller to Page**，理解它跟 fe05 的 `sap.ui.controllerExtensions` 是同一套機制，只是目標 Controller 類別名稱要交給工具自動偵測（可能是新式 `sap.fe.templates.*` 也可能是舊式 `sap.suite.ui.generic.template.*`）
- 能操作 **Add Custom Table Column**，理解 Fragment-based View Extension 如何透過穩定 ID + `targetAggregation / index` 疊加到標準 App 的既有控制項上
- 知道「**Enhance UI Annotation**」在標準 App 情境下的真正做法：**Change Table Columns**（既有欄位顯示切換）vs **Add Custom Table Column**（Fragment 自建欄位），兩者都不是直接編輯對方的 Annotation 定義

物件清單

這一課建立了一個全新的、獨立的前端專案（跟 fe01 ~ fe05 疊加同一個 `fe01_connection_test` 不同），沒有新增或修改任何 ABAP 物件（不能改，也沒有需要改）：

檔案	說明
<code>fe06.travel.ext/webapp/manifest.appdescr_variant</code>	Application Variant 描述檔， reference 指向 <code>sap.demo.travel.v2</code>
<code>fe06.travel.ext/webapp/changes/coding/TravelListExt.js</code>	Controller Extension（ <code>sap.ui.controllerExtensions</code> ），加了 <code>onInit</code> <code>MessageToast</code>
<code>fe06.travel.ext/webapp/changes/id_..._codeExt.change</code>	上面那個 Controller Extension 的變更描述檔
<code>fe06.travel.ext/webapp/changes/fragments/CustomColumnHeader.fragment.xml</code>	新增表格欄位的標題 Fragment
<code>fe06.travel.ext/webapp/changes/fragments/CustomColumnCell.fragment.xml</code>	新增表格欄位的儲存格 Fragment
<code>fe06.travel.ext/webapp/changes/id_..._addXML.change</code> （兩份）	上面兩個 Fragment 各自的變更描述檔

動手練習

輪到你了：

1. 切到 Object Page (在 Adaptation Editor 用 **Navigation** 模式點進任一筆 Travel) , 比較它的 Quick Actions 清單 (**Add Header Field** / **Add Custom Section**) 跟 List Report 有什麼不同 , 試著執行一個 **Add Custom Section** , 看產生出來的 Fragment 結構跟這一課的表格欄位 Fragment 有什麼差異
2. 把 `CustomColumnCell.fragment.xml` 裡的 `<Text id="cell-text-b4ca15a5" text="Sample data" />` 改成真的顯示資料 (提示 : Fragment 是標準 UI5 View 語法 , 可以用 `{}` Binding 語法綁定目前這一系列的 Context 屬性 , 例如 `text="{TotalPrice}"` ——想一想這個 Binding Context 從哪裡來、跟表格本身綁定的 OData Entity 有什麼關係)
3. 想一想 (不用真的做) : 這一課完全沒有測試 **Deploy** (`npm run deploy` 或 VS Code 的 Deploy 指令) 。Lecture 裡提到官方文件明講「不能部署到 ABAP for Cloud Development 套件」——如果你真的執行 Deploy , 你預期會在哪一步卡住 ? 錯誤訊息可能會提到什麼關鍵字 ? (提示 : 想一想這系統的套件屬性 , 回頭看 RAP Cloud 課程學過的「ABAP Cloud 套件」概念)

驗證方式

`npm run start-editor` 本機執行 + 瀏覽器截圖確認 , 這一課全程用真人截圖 + 即時互動的方式驗證 (Adaptation Editor 本質上是視覺化工具 , 沒有無頭驗證的空間) :

1. **Controller Extension** : 已截圖確認——重新整理 Adaptation Editor 預覽 , 進入 List Report 時跳出 `customer.fe06.travel.TravelListExt` 已掛上標準 **Demo Travel App** 的 `MessageToast`
2. **Fragment 擴充** : 已確認硬碟上正確產生 `CustomColumnHeader.fragment.xml` / `CustomColumnCell.fragment.xml` 與對應的 `.change` 檔案 , `selector.id` 正確指向標準 App 的 `responsiveTable` 控制項 ; 這一課沒有進一步截圖確認「New column」欄位在畫面上實際顯示出來的樣子 (留給讀者在動手練習第 2 題順便驗證)
3. **Add Custom Table Column 的前置條件** : 已截圖確認 , 未載入資料時該按鈕灰階 , tooltip 正確說明原因 ; 按 Go 載入資料後按鈕轉為可點擊

思考題

1. 這一課發現 **Demo App for Travel V2** 底層其實是舊式 `sap.suite.ui.generic.template` (OData V2 Smart Template) , 不是新式 `sap.fe.templates` (OData V4) 。如果你今天要擴充的是一個更現代、真的用 `sap.fe.templates` 建的標準 App , 你覺得 Adaptation Editor 操作起來的體驗 (Quick Actions 清單、Controller Extension 掛的目標 Controller 名稱) 會不會不一樣 ? 為什麼工具本身要幫你自動偵測這件事 , 而不是讓你自己選 ?
2. **Change Table Columns** 只能調整「標準 App CDS View 已經曝露的既有欄位」的顯示與否 , 不能新增全新欄位。如果客戶的需求是「希望在這個標準 App 上看到一個 CDS View 完全沒有的全新業務欄位」(例如某個自訂計算邏輯的結果) , 你覺得除了 **Add Custom Table Column** (Fragment 塞假資料) , 有沒有更根本的做法能讓這個新欄位真的連結到後端資料 ? (提示 : 想一想 CLAUDE.md 提過的「DDIC Structure 的 Customer Include」這類官方預留的擴充點——但那是給我們自己的 CDS View 用的 , 標準 App 的 CDS View 有沒有類似的機制 , 可能需要另外查證)
3. 這一課的兩筆變更 (Controller Extension、Fragment) 都各自產生了獨立的 `.change` 檔案 , 而不是像 `manifest.appdescr_variant` 那樣全部塞進同一個陣列。你覺得這樣設計的理由是什麼 ? (提示 : 想一想多人協作、版本控制、之後要「刪除某一筆變更」的情境 , 一個變更一個檔案 vs 全部擠在同一個陣列裡 , 各自的優缺點)

答案

見 `fe06.travel.ext/webapp/manifest.appdescr_variant`、`fe06.travel.ext/webapp/changes/` 底下的所有檔案。沒有新增或修改任何 ABAP 物件 (也不能改 , **Demo App for Travel V2** 是 SAP 標準物件) ; 這是一個全新獨立建立的前端專案 , 不是疊加在 `fe01_connection_test` 上。實測結果 : Controller Extension 端對端驗證成功 (`MessageToast` 截圖為證)、Fragment 擴充的檔案結構已確認正確產生 , Deploy 未實測 (風險已記錄在 Lecture / 思考題) 。